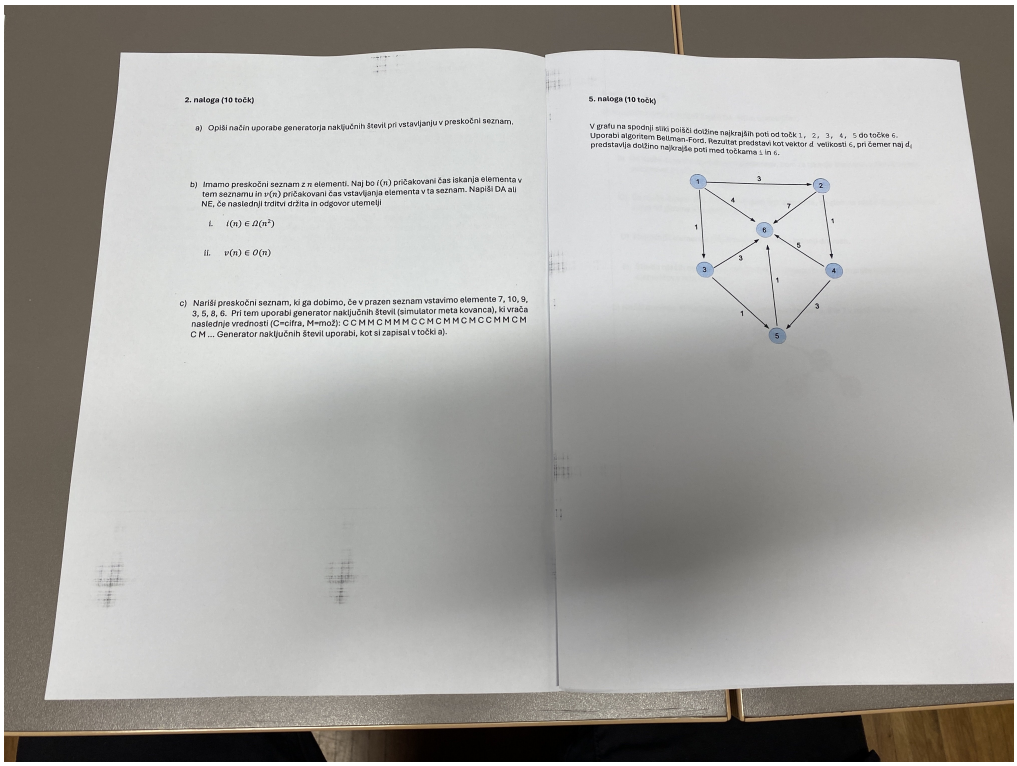
IMG_5395.jpg - preskočni seznam in Bellman-Ford



2. naloga (10 točk)

a) Opiši način uporabe generatorja naključnih števil pri vstavljanju v preskočni seznam.

b) Imamo preskočni seznam z n elementi. Naj bo $i(n)$ pričakovani čas iskanja elementa v tem seznamu in $v(n)$ pričakovani čas vstavljanja elementa v ta seznam. Napiši DA ali NE, če naslednji trditvi držita in odgovor utemelji.

i. $i(n) \in \Omega(n^2)$

ii. $v(n) \in O(n)$

c) Nariši preskočni seznam, ki ga dobimo, če v prazen seznam vstavimo elemente 7, 10, 9, 3, 5, 8, 6. Pri tem uporabi generator naključnih števil (simulator meta kovanca), ki vrača naslednje vrednosti (C=cifra, M=mož): C C M M C M M M C C M C M M C M C C M M C M C M ... Generator naključnih števil uporabi, kot si zapisal v točki a).

5. naloga (10 točk)

V grafu na spodnji sliki poišči dolžine najkrajših poti od točk $1, 2, 3, 4, 5$ do točke 6. Uporabi algoritem Bellman-Ford. Rezultat predstavi kot vektor d velikosti 6, pri čemer naj d_i predstavlja dolžino najkrajše poti med točkama i in 6.

Rešitev / postopek

2. naloga - preskočni seznam

a) Uporaba generatorja naključnih števil

Pri vstavljanju elementa določimo njegovo višino z meti kovanca:

- element je vedno na nivoju 0,
- dokler generator vrača `C`, element promoviramo za en nivo,
- ko vrne `M`, se ustavimo.

Predpostavka: `C = uspeh/promocija`, `M = stop`.

b) Trditve

Pričakovano iskanje in vstavljanje v preskočni seznam sta $\Theta(\log n)$.

- `i(n) ∈ Ω(n^2)` - NE. `log n` ne raste vsaj tako hitro kot `n^2`.
- `v(n) ∈ O(n)` - DA. Ker `log n ∈ O(n)`.

c) Vstavljanje `7, 10, 9, 3, 5, 8, 6`

Predpostavka za zaporedje metov: `C C M M C M M M C C M C M ...`

Višine:

- `7`: `C, C, M` -> nivoji 0,1,2
- `10`: `M` -> nivo 0

- 9: `C`, `M` -> nivoji 0,1
- 3: `M` -> nivo 0
- 5: `M` -> nivo 0
- 8: `C`, `C`, `M` -> nivoji 0,1,2
- 6: `C`, `M` -> nivoji 0,1

Končni preskočni seznam:

```
nivo 2: 7 -------- 8
nivo 1: 6 -- 7 -- 8 -- 9
nivo 0: 3 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
```

5. naloga - Bellman-Ford

Prebrane povezave:

- `1->2 (3)`, `1->3 (1)`, `1->6 (4)`
- `2->4 (1)`, `2->6 (7)`
- `3->5 (1)`, `3->6 (3)`
- `4->5 (3)`, `4->6 (5)`
- `5->6 (1)`

Iščemo razdalje od vozlišč `1..6` do vozlišča `6`.

Najkrajše poti:

- `d_6 = 0`
- `d_5 = 1` po `5->6`
- `d_4 = 4` po `4->5->6`
- `d_3 = 2` po `3->5->6`
- `d_2 = 5` po `2->4->5->6`
- `d_1 = 3` po `1->3->5->6`

Vektor:

```
d = [3, 5, 2, 4, 1, 0]
```